Presenter Notes — Week 3 Design Review (10 sheets)

The story of this presentation in one sentence: *last review we showed an architecture; this week we show it running — real packets, real math, a real screen — and the physics that explains its behavior.*

Opening line (whoever starts): "Two weeks ago these were diagrams. Today the display shows a live vehicle: position decoded from real radio packets, an arrow from real navigation math, and a screen that moves every second. Each section opens with its architecture, then the working hardware and the code."

Sheet 01 — Cover

Read the subtitle, point at the status row in the title block: **29% complete against a 19% week-three target, 60 hours logged.** One line per owner from the table. Don't linger; the substance is ahead.

Sheet 02 — The Complete System on One Sheet ⭐ DEO

This is the map for everything that follows. Walk it in reading order:

1. **Left region, one vehicle's hardware:** GNSS module feeds the Heltec radio over UART; the radio drives the roof antenna; the Waveshare display sits a meter away; both share vehicle 12 V power (powered together deliberately).
2. **Right region, inside the display firmware:** my protocol chain (BLE link, decode, node table) writes the TrackerState contract; Josiah's chain (nav engine, renderer) reads it. One writer, one reader, one mutex.
3. **Bottom strip, the telecom view:** a 40 byte application payload, a managed-flooding network layer with AES-256, and a physical layer that decodes 15 dB below the noise floor.

Close: "Vehicle B is an identical copy. The symmetry is the design — no server, no master, no single point of failure."

Cheat: only custom code = the two chains on the right · they meet at exactly one struct · everything radio-hard ships pre-tested in Meshtastic.

Sheet 03 — Backend: the Data Spine ⭐ DEO

Zoom from the map into the firmware diagram. Three beats:

- **The chain:** ble_link connects directly to the radio's remembered address (a Meshtastic radio only advertises briefly at boot, so scanning alone could never reconnect), encrypts with Just Works pairing, subscribes to the notify "doorbell," and drains the message queue until the radio says empty.
- **The decode:** nanopb, generated from Meshtastic's own published schema, turns each binary frame into named fields. node_table stores both vehicles' fixes with timestamps and detects stale re-broadcasts.
- **The contract:** TrackerState v2 now carries speed, ages, and the GPS wall clock that drives the on-screen time.

Cheat: road-proven, not bench-proven: the link reconnected itself after every drop across four drive tests. Q: "What if BLE fails permanently?" → the identical protobuf stream is available over WiFi/TCP; decoder and UI would not change.

Sheet 04 — How the Pieces Connect in Code ⭐ DEO

Read the four boxes as one journey, left to right, top to bottom:

1. **ble_link.cpp** reads the GATT queue until a zero-length read says empty (one notify can mean many queued frames).
2. **proto_decode.cpp** decodes each frame; a Position becomes named fields.
3. **node_table.cpp** publishes the whole picture once per second — the ONE writer.
4. **ui_render.cpp** (Josiah's side, purple) copies a snapshot at 2 Hz, dead reckons it forward, and computes the arrow — the ONE reader.

The mutex boundary arrow is the entire team interface. The store functions take the lock internally, so forgetting the lock is impossible.

Cheat — the hardening list (all from real drives): wire speed is km/h and is converted at ingestion; GPS jitter under 4.5 km/h reads as zero; a bit-identical re-broadcast cannot reset the data age; the clock can never run backwards. Every item was a road-test bug first.

Sheet 05 — Frontend: From Struct to Arrow ⭐ JOSIAH

Flow first: snapshot → dead reckon → navigate → render. Then the two code fragments:

- **nav.c:** haversine for distance, initial bearing, and the arrow is simply bearing minus own course, wrapped to ±180. Zero means dead ahead.
- **dead_reckon.c:** project each fix forward along its course at its speed, capped at 120 seconds so a ghost freezes instead of sailing away.

Design points: heading comes from GPS course over ground because a magnetometer inside a steel cab is unreliable; below 5 mph the arrow becomes a north-up rose because course is noise when parked; every banner and chevron is a state machine on the struct. **12 of 12 native unit tests pass** — these files run on a laptop with no hardware, which is how the UI was built before the radio link existed.

Sheet 06 — The UI on Real Packets ⭐ JOSIAH

Let the photos speak, one line each:

- Driving view: partner 2.41 miles, CHANGE COURSE banner with the turn chevron, age 15 seconds.
- Stopped: the automatic north-up rose fallback.
- Close range: 0.2 miles with the oversized arrow.
- Status page: live ME and PTR coordinates, the debugging view.

Key sentence: "There is no simulator in these photos. Every value arrived by radio and was decoded by our stack." Second point: the reflective panel gets *sharper* in more light — built for a dashboard in the sun.

Sheet 07 — Radio and PHY ⭐ VICTOR

Hardware diagram, then the configuration that three road tests produced:

- LONG_FAST preset: SF11, 250 kHz, FEC 4/5, 915 MHz ISM, AES-256 private channel at full position precision.
- Smart broadcast every 10 s or 20 m moving, 60 s parked; GPS refresh also 10 s so every broadcast carries a genuinely new fix (mismatch here was a real bug: 30 s GPS refresh made 10 s broadcasts repeat stale fixes).
- The OLED is the free field debugger: satellites, nodes, channel utilization, last RSSI.

Cheat — provisioning: radio #2 joins with the exact channel URL (it embeds the AES key). Two documented traps: reboot-triggering settings can silently revert when written over USB (the port close resets the chip mid save — always read back and verify), and identical boards demand explicit upload ports.

Sheet 08 — Real Time versus 10 Seconds ⭐ VICTOR (the math sheet)

Walk the five equations in order; each one earns the next:

1. **Airtime:** symbol time is 2^SF over bandwidth. At SF11 each symbol is 2048 chips ≈ 8.2 ms, and a position packet costs ≈ 0.5 s of air.
2. **Utilization:** two vehicles at 1 Hz would occupy 100% of the channel — collapse. At 10 s they occupy 10%, healthy, under the 25% cap.
3. **Sensitivity:** the same spreading factor buys −131 dBm, which is 15 dB *below* thermal noise. This is where the kilometers come from.
4. **Latency:** information reaches the driver ≈ 1 s after it exists. The system IS real time in information delay; 10 s is the *sampling* period.
5. **Dead reckoning:** the display projects both vehicles forward every second between fixes, capped at the 120 s stale horizon.

The verdict, verbatim if you like: "Rate and range are the same currency, and we spent it on range." If asked for precedent: marine AIS beacons every 2 to 30 seconds and every chartplotter animates between reports.

Sheet 09 — Integration Constraints ⭐ CANNON

Frame it as provenance: "None of these are theoretical. Every card was discovered on our bench or on our roads, and each one shaped the design." Then pick two or three to tell as stories:

- **GPS:** indoor multipath faked 1–3 km/h of phantom speed until the deadband; the fix-rate/broadcast-rate mismatch made 10 s broadcasts repeat stale fixes.
- **BLE:** the radio advertises only from boot until first disconnect, hence the ignition-on rule: both units power up together.
- **Power:** cheap 12 V USB adapters cut off on the radio's low draw.

Close with the test sequence: provision radio 2 → bench two-radio decode → two-vehicle drive test → range characterization. Each constraint card becomes protocol line items.

Sheet 10 — Status and Next

Read the four stat tiles. Then the two lists briefly. End on the closing bar, verbatim — it's the best line in the deck:

> "One radio still talks to itself. But every layer between a GPS fix and the driver's arrow is now real, tested code. Radio number two turns this into a tracker."

Q&A ammunition (whole team)

- **Why not 1 Hz updates?** Sheet 08, equations 1 and 2. Airtime is the cost; two vehicles at 1 Hz exceed the channel. Dead reckoning gives 1 Hz motion anyway.
- **What if a packet is lost?** Nothing breaks. A newer position supersedes an older one; the age counts up; the next broadcast heals it. Data age is the reliability metric, shown to the driver.
- **Why no magnetometer?** Steel cab. Course over ground is accurate above 5 mph, and a driving use case is usually moving; parked mode uses the rose.
- **Is the data secure?** AES-256 on a private channel; an eavesdropper sees noise-like bursts. The channel URL embedding the key never leaves the team.
- **How do you know the math is right?** 12 native unit tests run on every build, plus the physics on sheet 08 matches the measured behavior (0.5 s airtime, utilization readouts on the radio's own OLED).